The following list shows some of the names that might be used to define a data type: character, varchar, boolean, integer, real, date, time. In this chapter these will be written in lower case, but you might see them written in upper case in other sources.

TASK 11.03

For the database defined in Worked Example 11.02, complete the DDL for creating the four tables. Use varchar(8) for BookingID, integer for NumberOfMembers, date for Date, character for Headlining and varchar(25) for all other data.

Data Manipulation Language (DML)

There are three categories of use for Data Manipulation Language (DML)

- The insertion of data into the tables when the database is created
- The modification or removal of data in the database
- The reading of data stored in the database

The following illustrate the two possible ways that SQL can be written to populate a table with data:

```
INSERT INTO Band ('ComputerKidz', 5);
INSERT INTO Band-Booking (BandName, BookingID)
VALUES ('ComputerKidz', '2016/023');
```

The first example shows a simpler version that can be used if the order of the attributes is known. The second shows the safer method; the attributes are defined then the values are listed. The following are some points to note.

- Parentheses are used in both versions.
- A separate INSERT command has to be used for each tuple in the table.
- There is an order defined for the attributes.
- Although the SQL will have a list of INSERT commands the subsequent use of the table has no concept of the tuples being ordered.

The main use of DML is to obtain data from a database using a query. A query always starts with the `SELECT` command. Two examples are:

The simplest form for a query has the attributes for which values are to be listed as output identified after SELECT and the table name identified after FROM. For example:

```
SELECT BandName FROM Band;
```

Note that the components of the query are separated by spaces.

The Band table only has two attributes. To list the values for both there are two options:

```
SELECT BandName, NumberOfMembers
FROM Band;
```

or

```
SELECT * FROM Band;
```

which uses * to indicate all attributes. Note that in the first example the attributes are separated by commas but no parentheses are needed.

It is possible to include instructions in the SQL to control the presentation of the output. The following uses ORDER BY to ensure that the output is sorted to show the data with the band names in alphabetical order.

```
SELECT BandName, NumberOfMembers
FROM Band
ORDER BY BandName;
```

In this query there is no question of duplicate entries because BandName is the primary key of the BandName table. However, in the Band-Booking table an individual value for BandName will occur many times. If a query were being used to find which bands already had a booking there would be repeated names in the output. This can be prevented by the use of GROUP BY as shown here:

```
SELECT BandName
FROM Band-Booking
GROUP BY BandName;
```

An extension of the control of the output from a query is to include a condition to limit the selected data. This is provided by a WHERE clause. The following are examples:

```
SELECT BandName
FROM Band-Booking
WHERE Headlining = 'Y'
GROUP BY BandName;
```

which produces a single output for each band that has headlined. Note how a query can have several component parts which are best presented on separate lines.

```
SELECT BandName, NumberOfMembers
FROM Band
WHERE NumberOfMembers > 2
ORDER BY BandName;
```


which excludes any duo bands.

It is possible to qualify the SELECT statement by using a function. SUM, COUNT and AVG are examples of functions that work on data held in several tuples for a particular attribute and return one value. For this reason, these functions are called aggregate functions. As an example, the following code displays the number of members in a band:

```
SELECT Count(*)
FROM Band;
```

This is a special case because there is no need to specify the attribute. An example using a specific attribute would be:

```
SELECT AVG(NumberOfMembers)
FROM Band;
```

another example is:

```
SELECT SUM(NumberOfMembers)
FROM Band;
```

A query can be based on a 'join condition' between data in two tables. The most frequently used is an inner join which is illustrated by:

```
SELECT VenueName, Date
FROM Booking
WHERE Band-Booking.BookingID = Booking.BookingID
AND Band-Booking.BandName = 'ComputerKidz';
```

The SQL uses the full definitive name for each attribute with the table name and attribute name separated by a dot. The query contains two conditions. The way that the query works is as follows.

- The Band-Booking table is searched for instances where the BandName is ComputerKidz.
- For each instance the BookingID is noted.

- Then there is a search of the Booking table to find the examples of tuples having this value for BookingID.
- For each one found the VenueName and Date are presented in the output.

Some versions of SQL require the explicit use of INNER JOIN. The following is a possible generic syntax:

```
SELECT table1.column1, table2.column2...
FROM table1
INNER JOIN table2
ON table1.common_field = table2.common_field;
```

The other use of DML is to modify the data stored in the database. The UPDATE command is used to change the data. If the band ComputerKidz recruited an extra member the following SQL would make the change needed.

```
UPDATE Band
SET NumberOfMembers = 6
WHERE BandName = 'ComputerKidz';
```

Note the use of the WHERE clause. If you forgot to include this the UPDATE command would change the number of band members to 6 for all of the bands.

The DELETE command is used to remove data from the database. This has to be done with care. If the ITWizz band decided to disband the following SQL would remove the name from the database.

```
DELETE FROM Band-Booking
WHERE BandName = 'ITWizz';
DELETE FROM Band
WHERE BandName = 'ITWizz';
```

Note that if an attempt was made to carry out the deletion from Band first there would be an error. This is because BandName is a foreign key in Band-Booking. Any entry for BandName in Band-Booking must have a corresponding value in Band.

Reflection Point:

Did you find normalisation difficult? It would be surprising if you didn't. Are you going to get as much practice as possible? There are many questions from previous exam papers that contain examples to try.

Summary

- A database offers improved methods for ensuring data integrity compared to a file-based approach.
- A relational database comprises tables of a special type; each table has a primary key and may contain foreign keys.
- Entity–relationship modelling is a top-down approach to database design.
- Normalisation is a database design method that starts with a collection of attributes and converts them into first normal form then into second normal form and, finally, into third normal form.
- A database architecture provides, for the user, a conceptual level interface to the stored data.
- Features provided by a database management system (DBMS) include: a data dictionary, indexing capability, control of user access rights and backup procedures.
- Structured Query Language (SQL) includes data definition language (DDL) commands for establishing a database and data manipulation language (DML) commands for creating queries.

Exam-style Questions

1 **a** A relational database has been created to store data about subjects that students are studying. The following is a selection of some data stored in one of the tables. The data represents the student's name, the personal tutor group, the personal tutor, the subject studied, the level of study and the subject teacher but there are some data missing:

Xiangfei	3	MUB	Computing	A	DER
Xiangfei	3	MUB	Maths	A	BNN
Xiangfei	3	MUB	Physics	AS	DAB
Mahesh	2	BAR	History	AS	IJM
Mahesh	2	BAR	Geography	AS	CAB

i Define the terms used to describe the components in a relational database table using examples from this table. [2]

ii If this represented all of the data, it would have been impossible to create this table. Identify what has not been shown here and must have been defined to allow the creation as a relational database table? Explain your answer and suggest a solution to the problem. [4]

iii Is this table in first normal form (1NF)? Explain your reason. [2]

b It has been suggested that the database design could be improved. The design suggested contains the following two tables:

Student(StudentName, TutorGroup, Tutor)

StudentSubject(StudentName, Subject, Level, SubjectTeacher)

i Identify features of this design which are characteristic of a relational database. [3]

ii Explain why the use of StudentName here is a potential problem. [2]

iii Explain why the Student table is not in third normal form (3NF). [2]

2 Consider the following scenario:

A company provides catering services for clients who need special-occasion, celebratory dinners. For each dinner, a number of dishes are to be offered. The dinner will be held at a venue. The company will provide staff to serve the meals at the venue.

The company needs a database to store data related to this business activity.

a An entity–relationship model is to be created as the first step in a database design. Identify a list of entities. [4]

b Identify pairs of entities where there is a direct relationship between them. [4]

c For each pair of entities, draw the relationship and justify the choice of cardinality illustrated by the representation. [6]

3 Consider the following booking form used by a travel agency.

Booking Number 00453

Hotel: Esplanade
Colwyn Bay
North Wales

Rating: ***

Date	Room type	Number of rooms	Room rate
23/06/2016	Front-facing double	2	$80
23/06/2016	Rear-facing double	1	$65
24/06/2016	Front-facing double	2	$80

a Identify an unnormalised list of attributes using the data shown in this form. Make sure that you distinguish between the repeating and non-repeating attributes. [5]

b Demonstrate the conversion of the data to first normal form (1NF). The design of two tables should be defined with the keys identified. [3]

c Identify the appropriate table and demonstrate the conversion of the table to two tables in second normal form (2NF). Explain your choice of table to modify. Explain your identification of the keys for these two new tables. [5]

d Identify which part of your design is not in Third Normal Form (3NF). [2]

4 A small database is to be created with the following three tables:

STUDENT(StudentID, StudentName, StudentOtherName, DateOfbirth)

SUBJECT(SubjectName, SubjectTeacher)

TUTORIAL(StudentID(fk), Subjectname(fk), WeekNumber, Day, PeriodNumber)

a Using the appropriate datatypes from the following list:

CHARACTER, VARCHAR2, Boolean, INTEGER, REAL, DATE

Write the SQL scripts to create two of the tables using the CREATE TABLE command. Do not at this stage identify any keys. [5]

b Assuming that all three tables have been created, write the SQL scripts to assign the primary key in the SUBJECT table and the two foreign keys in the TUTORIAL table. [3]

c Write the SQL script that will list all of the student names in age order. [5]

d There is an aspect of the design of the tables that could cause problems. Explain this problem. [2]

5 A school stores a large amount of data. This includes student attendance, qualification and contact details. The school's software uses a file-based approach to store this data.

a The school is considering changing to a DBMS.

i State what DBMS stands for. [1]

ii Describe **two** ways in which the database Administrator (DBA) could use the DBMS software to ensure the security of the student data. [4]

iii A feature of the DBMS software is a query processor.

Describe how the school secretary could use this software. [2]

iv The DBMS has replaced software that used a file-based approach with a relational database.

Describe how using a relational database has overcome the previous problems associated with a file-based approach. [3]

b The database design has three tables to store the classes that students attend.

```
STUDENT (StudentID, FirstName, LastName, Year, TutorGroup)
CLASS (ClassID, Subject)
CLASS -GROUP (StudentID, ClassID)
```

Primary keys are not shown.

There is a one-to-many relationship between CLASS and CLASS –GROUP.

i Describe how this relationship is implemented. [2]

ii Describe the relationship between CLASS –GROUP and STUDENT. [1]

iii Write an SQL script to display the `StudentID` and `FirstName` of all students who are in the tutor group 10B. Display the list in alphabetical order of `LastName`. [4]

iv Write an SQL script to display the `LastName` of all students who attend the class whose `ClasstID` is CS1 [4]

Cambridge International AS & A level Computer Science 9608 paper 11 Q8 June 2016

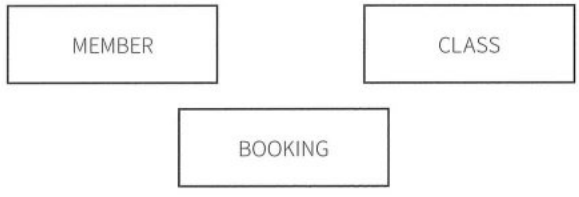
6 A health club offers classes to its members. A member needs to book into each class in advance.

a The health club employs a programmer to update the class booking system. The programmer has to decide how to store the records. The choice is between using a relational database or a file-based approach.

Give **three** reasons why the programmer should use a relational database. [6]

b The programmer decides to use three tables: MEMBER, BOOKING and CLASS.

Complete the entity–relationship (E–R) diagram to show the relationships between these tables.



[2]

c The `Class` table has primary key `Class ID` and stores the following data:

ClassID	Description	StartDate	ClassTime	NoOfSessions	Adultsonly
DAY01	Yoga beginners	12/01/2016	11:00	5	TRUE
EVE02	Yoga beginners	12/01/2016	19:00	5	FALSE
DAY16	Circuits	30/06/2016	10:30	4	FALSE

Write an SQL script to create the CLASS table. [6]

Cambridge international AS & A Level Computer Science 9608 paper 12 Q9 November 2016